

Q.Explain Keywords, Identifiers, Literals, Operators?

Ans.

Keywords

Keywords are **reserved words** in Python that have a predefined meaning and cannot be used as identifiers.

Examples:

if, else, for, while, def, class, True, False

```
if x > 10:
    print("Greater")
```

Identifiers

Identifiers are names given to variables, functions, classes, or objects to identify them in a program.

Rules:

- Can contain letters, digits, and underscore
- Must not start with a digit
- Cannot be a keyword

Examples:

```
total = 50
student_name = "Amit"
```

Literals

Literals are **fixed constant values** used in a program.

Types with examples:

- Numeric: 10, 3.5
- String: "Python"
- Boolean: True, False
- Special: None

Example:

```
x = 10
name = "Python"
flag = True
```

Operators

Operators are symbols used to perform operations on variables and values.

Examples:

- Arithmetic: +, -, *, /
- Relational: >, <, ==
- Logical: and, or, not

```
a = 10
b = 5
sum = a + b
```

Q. Explain Data Types- Number, Strings, Lists, Tuples, Dictionaries, Sets

Ans.

1. Number Data Type

The Number data type is used to store numeric values. Python supports integers, floating-point numbers, and complex numbers.

Examples:

```
a = 10    # int
b = 3.14  # float
c = 2 + 3j # complex
```

2. String Data Type

A **String** is a sequence of characters enclosed in single quotes, double quotes, or triple quotes. Strings are **immutable**.

Examples:

```
name = "Python"
msg = 'Hello'
```

3. List Data Type

A **List** is an ordered, mutable collection of elements. It can store different data types.

Examples:

```
marks = [85, 90, 78]
data = [1, "Python", 3.5]
```

4. Tuple Data Type

A **Tuple** is an ordered collection of elements that is **immutable** (cannot be changed after creation).

Examples:

```
t1 = (10, 20, 30)
t2 = ("A", "B", "C")
```

5. Dictionary Data Type

A **Dictionary** stores data in **key-value pairs**. Keys must be unique and immutable.

Examples:

```
student = { "roll": 1, "name": "Amit", "marks": 80 }
```

6. Set Data Type

A **Set** is an unordered collection of **unique elements**. Duplicate values are not allowed.

Examples:

```
s = {1, 2, 3, 3}
print(s)  # {1, 2, 3}
```

Q. Explain Control flow- if, else, elif

Ans.

Control flow statements are used to control the execution of a program based on conditions.

1.if Statement

The if statement executes a block of code only when the condition is true.

Example:

```
x = 10
```

```
if x > 5:
```

```
    print("x is greater than 5")
```

2. if-else Statement

The else block executes **when the if condition is false**.

Example:

```
x = 3
```

```
if x > 5:
```

```
    print("Greater")
```

```
else:
```

```
    print("Smaller")
```

3. elif Statement

The elif (else if) statement is used to **check multiple conditions**. It is executed when the previous condition is false and its condition is true.

Example:

```
marks = 75
```

```
if marks >= 80:
```

```
    print("Distinction")
```

```
elif marks >= 60:
```

```
    print("First Class")
```

```
else:
```

```
    print("Pass")
```

Q. Explain Loops- while, for, continue, break.

Ans.

Loops are used to execute a block of code repeatedly until a given condition is satisfied.

1. while Loop

The while loop executes a block of code as long as the condition is true.

Example:

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i += 1
```

2. for Loop

The for loop is used to **iterate over a sequence** such as a list, tuple, string, or range.

Example:

```
for i in range(1, 6):
```

```
    print(i)
```

3. continue Statement

The continue statement is used to **skip the current iteration** of the loop and continue with the next iteration.

Example:

```
for i in range(1, 6):
```

```
    if i == 3:
```

```
        continue
```

```
    print(i)
```

4. break Statement

The break statement is used to **terminate the loop immediately** when a condition is met.

Example:

```
for i in range(1, 6):
```

```
    if i == 4:
```

```
        break
```

```
    print(i)
```

Q. Explain Comprehensions on List, Tuple Dictionaries

Ans.

Comprehensions provide a **short and concise way to create collections** using a single line of code.

1. List Comprehension

List comprehension is used to **create a list** from an existing iterable based on an expression and condition.

Syntax:

[expression for item in iterable if condition]

Example:

```
squares = [x*x for x in range(1, 6)]
```

2. Tuple Comprehension

Tuple comprehension is similar to list comprehension but **produces a tuple using the tuple() function** (internally uses a generator).

Example:

```
t = tuple(x*x for x in range(1, 6))
```

3. Dictionary Comprehension

Dictionary comprehension is used to **create dictionaries with key-value pairs** in a compact form.

Syntax:

```
{key: value for item in iterable if condition}
```

Example:

```
square_dict = {x: x*x for x in range(1, 6)}
```

Q. Explain lambda function with example

Ans.

Lambda Function

A **lambda function** is a **small anonymous function** defined using the lambda keyword.

It can have **any number of arguments but only one expression**.

Syntax:

lambda arguments : expression

Example:

```
add = lambda a, b: a + b
```

```
print(add(5, 3))
```

Output:

8

Key Points:

No function name (anonymous)

Used for short, simple operations

Commonly used with functions like map(), filter(), reduce()

Q. Explain Decorators and Generators

Ans.

Decorators

A **decorator** is a function that **modifies the behavior of another function** without changing its source code.

It is implemented using the @ symbol.

Example:

```
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper
```

```
@my_decorator
```

```
def show():
    print("Hello Python")
```

Generators

A **generator** is a function that returns an **iterator** and produces values **one at a time** using the yield keyword.

Example:

```
def my_generator():
    yield 1
    yield 2
    yield 3
```

```
for i in my_generator():
```

```
    print(i)
```

Key Points:

- Uses yield instead of return
- Saves memory
- Generates values **on demand**

Q. Explain Modules: Module basic usage, Creating, importing modules.

Ans.

A module is a file that contains Python code such as functions, variables, and classes.

It helps in code reusability, organization, and maintenance.

1. Basic Usage of Module

Modules allow us to **reuse code** written in another file without rewriting it.

Example (using built-in module):

```
import math
print(math.sqrt(16))
```

2. Creating a Module

A module is created by **saving Python code in a .py file.**

Example:

Create a file named mymodule.py

```
def add(a, b):
    return a + b
```

3. Importing a Module

There are different ways to import modules:

a) Import entire module

```
import mymodule
print(mymodule.add(2, 3))
```

b) Import specific function

```
from mymodule import add
print(add(4, 5))
```

c) Import with alias

```
import mymodule as m
print(m.add(6, 7))
```

Q. Explain Python built-in modules - math, random, datetime

Ans.

Python provides **built-in modules** that contain predefined functions and classes to perform common tasks.

1. math Module

The math module provides **mathematical functions and constants.**

Common functions:

- `sqrt()` – square root
- `pow()` – power

- `factorial()` – factorial
- `pi` – value of π

Example:

```
import math
print(math.sqrt(16))
print(math.pi)
```

2. random Module

The random module is used to **generate random numbers.**

Common functions:

- `randint(a, b)` – random integer between a and b
- `random()` – random float between 0 and 1
- `choice()` – random element from a list

Example:

```
import random
print(random.randint(1, 10))
```

3. datetime Module

The datetime module is used to **work with date and time.**

Common functions:

- `date.today()` – current date
- `datetime.now()` – current date and time

Example:

```
import datetime
print(datetime.date.today())
print(datetime.datetime.now())
```

- `math` → mathematical calculations
- `random` → random number generation
- `datetime` → date and time handling

Q. Explain User defined modules and packages
Ans.

User Defined Modules

A **user defined module** is a Python file created by the programmer to store **functions, variables, or classes** for reuse.

Creating a User Defined Module

Create a file with .py extension.

Example: calc.py

```
def add(a, b):  
    return a + b
```

```
def sub(a, b):  
    return a - b
```

User Defined Packages

A **package** is a **collection of related modules** organized in a directory.

It helps in **better structuring and management of large programs**.

Creating a Package

- Create a folder (package name)
- Add modules inside it
- Include `__init__.py` file (to mark directory as a package)

Structure:

```
mypackage/  
├── __init__.py  
├── module1.py  
└── module2.py
```

Using a Package

```
from mypackage import module1
```

Conclusion

- Module → Single Python file
- Package → Collection of modules
- Both support **code reusability and modular programming**

Q. Explain Exception Handling
Ans.

Exception handling is a mechanism used to **handle runtime errors** so that the normal flow of the program is not interrupted.

Python uses try, except, else, and finally blocks for exception handling.

Basic Syntax

try:

 # code that may cause exception

except ExceptionName:

 # code to handle exception

Example

try:

 a = int(input("Enter number: "))

 b = int(input("Enter number: "))

 print(a / b)

except ZeroDivisionError:

 print("Division by zero not allowed")

except ValueError:

 print("Invalid input")

finally:

 print("Program Ended")

Key Points

- try → code that may cause an error
- except → handles the error
- else → executes if no exception occurs
- finally → always executes
- Improves **program reliability**

Q. Explain Constructors & destructors

Ans.

Constructor

A **constructor** is a special method used to **initialize objects** when they are created.

In Python, the constructor is defined using the `__init__()` method.

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
s1 = Student("Amit", 20)
print(s1.name, s1.age)
```

Output:

Amit 20

Key Points:

- Automatically called when an object is created
- Used to **initialize object attributes**

Destructor

A **destructor** is a special method that is **called when an object is destroyed**.

In Python, it is defined using the `__del__()` method.

Example:

```
class Student:
    def __del__(self):
        print("Object destroyed")
```

```
s1 = Student()
del s1
```

Output:

Object destroyed

Key Points:

- Called when object goes out of scope or is deleted
- Used to **release resources**

Conclusion:

- `__init__()` → constructor, initializes objects
- `__del__()` → destructor, cleans up before object is destroyed

Q. Explain Inheritance, super class, method overriding

Ans.

Inheritance

Inheritance allows a **class (child)** to **acquire properties and methods of another class (parent/super class)**.

It promotes **code reusability**.

Example:

```
class Parent:
    def greet(self):
        print("Hello from Parent")
```

```
class Child(Parent):
    pass
```

```
c = Child()
c.greet()
```

Output:

Hello from Parent

Super Class

A **super class** (parent class) is the class whose **properties and methods are inherited** by another class.

The `super()` function is used to **access parent class methods**.

Example:

```
class Parent:
    def greet(self):
        print("Hello from Parent")
```

```
class Child(Parent):
    def greet_child(self):
        super().greet()
        print("Hello from Child")
```

```
c = Child()
c.greet_child()
```

Output:

Hello from Parent

Hello from Child

Method Overriding

Method overriding occurs when a **child class redefines a method of the parent class** with the same name.

The child's method **replaces the parent's method** when called by a child object.

Example:

```
class Parent:
    def greet(self):
        print("Hello from Parent")
```

```
class Child(Parent):
    def greet(self):
        print("Hello from Child")
```

```
c = Child()
```

```
c.greet()
```

Output:

Hello from Child

Conclusion:

- **Inheritance** → child class reuses parent class
- **Super class** → parent class, can be accessed via `super()`
- **Method overriding** → child class redefines parent method

Q. Explain Static and Class methods

Ans.

Static Methods

A **static method** is a method that **belongs to the class but does not access instance or class attributes**.

It is defined using the `@staticmethod` decorator.

Example:

```
class Math:
    @staticmethod
    def add(a, b):
        return a + b
```

```
print(Math.add(5, 3))
```

Output:

8

Key Points:

- Cannot access `self` or `cls`
- Called using **Class name**

Class Methods

A **class method** is a method that **can access class attributes but not instance attributes**.

It is defined using the `@classmethod` decorator and takes `cls` as the first argument.

Example:

```
class Student:
    school = "ABC School"
```

```
@classmethod
def show_school(cls):
    print(cls.school)
```

```
Student.show_school()
```

Output:

ABC School

Key Points:

- Can access **class variables**
- Called using **Class name or object**
- Commonly used for **factory methods**

Conclusion:

- **Static method** → utility function, no access to class/instance variables
- **Class method** → access class variables, uses `cls`

Q. Explain Delegation and containership

Ans.

Delegation

Delegation is a technique where an object **handles a request by passing it to another associated object**.

It helps in **reusing functionality without inheritance**.

Example:

```
class Engine:
    def start(self):
        print("Engine started")
```

```
class Car:
    def __init__(self):
        self.engine = Engine() # delegation
    def start(self):
        self.engine.start() # delegate task
```

```
c = Car()
c.start()
Output:
Engine started
```

Containership (Composition)

Containership/Composition is when a class **contains objects of other classes as its members**.

It represents a “**has-a**” **relationship**.

Example:

```
class Battery:
    def charge(self):
        print("Battery charging")

class Phone:
    def __init__(self):
        self.battery = Battery() # containment
    def charge_phone(self):
        self.battery.charge()

p = Phone()
p.charge_phone()
```

Output:

Copy code
Battery charging

Conclusion:

- Delegation → object **passes tasks** to another object
- Containership → object **contains other objects as members**
- Both help in **code reuse without inheritance**

Q. Explain Python Regular Expression

Ans.

A **regular expression (regex)** is a sequence of characters that defines a **search pattern**.

It is used for **pattern matching, searching, and text manipulation**.

Python Module: re is used to work with regular expressions.

Basic Functions

1. **re.match()** – Checks for a match at the **beginning** of a string
2. **re.search()** – Searches for a pattern **anywhere** in the string

3. **re.findall()** – Returns a **list of all matches**
4. **re.sub()** – Replaces **pattern matches** with a new string

Example:

```
import re

pattern = r"\d+" # matches one or more digits
text = "My number is 1234"
result = re.findall(pattern, text)
print(result)
```

Output:

['1234']

Key Points

- \d → matches any digit
- \w → matches any alphanumeric character
- + → matches **one or more repetitions**
- Regex is used for **validation, parsing, and text extraction**

Q. Explain Pattern matching and searching using regex in python

Ans.

Python's re module provides functions to find patterns in strings using regular expressions.

Pattern Matching

- Checks if a string **matches a pattern from the beginning**.
- Function: re.match(pattern, string)

Example:

```
import re
text = "Hello123"
pattern = r"\w+"
match = re.match(pattern, text)
if match:
    print("Matched:", match.group())
```

Output:

Matched: Hello123

Searching

- Searches for a pattern **anywhere in the string**.
- Function: `re.search(pattern, string)`

Example:

```
import re
text = "My number is 1234"
pattern = r"\d+"
search = re.search(pattern, text)
if search:
    print("Found:", search.group())
```

Output:

Found: 1234

Key Points

- `re.match()` → matches **from start**
- `re.search()` → finds **any occurrence**
- Both return a **match object**, `.group()` gives the matched string

Q. Explain Real time parsing of data using regex

Ans.

Real-time parsing involves extracting specific information from data streams or input as it arrives.

Python's `re` module allows pattern matching **on-the-fly**, making it suitable for tasks like log monitoring, form validation, or text extraction.

Example: Extracting Emails from a Stream

```
import re
```

```
data_stream = ["Contact: john@example.com",
               "Reach: alice@mail.com", "No email here"]
```

```
pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
```

```
for line in data_stream:
```

```
    match = re.findall(pattern, line)
```

```
    if match:
```

```
        print("Found:", match)
```

Output:

Found: ['john@example.com']

Found: ['alice@mail.com']

Q. Explain Applications of Regex-Password, email, URL validation

Ans.

Password Validation

Regex can be used to **validate passwords** based on rules like:

- Minimum length
- At least one uppercase letter
- At least one digit
- At least one special character

Example:

```
import re
```

```
pattern = r'^(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$'
```

```
password = "Passw0rd@"
```

```
if re.match(pattern, password):
```

```
    print("Valid Password")
```

```
else:
```

```
    print("Invalid Password")
```

Email Validation

Regex is used to **check if an email is in the correct format** (username@domain.extension).

Example:

```
pattern = r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'
```

```
email = "john@example.com"
```

```
if re.match(pattern, email):
```

```
    print("Valid Email")
```

```
else:
```

```
    print("Invalid Email")
```

URL Validation

Regex helps **verify URLs** to ensure they follow the correct structure (http/https, domain, optional path).

Example:

```
pattern = r'https?://[\w.-]+'
```

```
url = "https://www.example.com"
```

```
if re.match(pattern, url):
```

```
    print("Valid URL")
```

```
else:
```

```
    print("Invalid URL")
```

Q. Explain Multithreading

Ans.

Multithreading allows a Python program to run multiple threads (lightweight processes) concurrently within the same program. It is mainly used to improve performance in I/O-bound tasks like file operations, network communication, or user input.

Creating Threads

Python provides the threading module to work with threads.

Example: Using Thread with target function
import threading

```
def print_numbers():
    for i in range(1, 6):
        print(i)

# Create a thread
t1 = threading.Thread(target=print_numbers)
t1.start() # Start thread
t1.join() # Wait for thread to finish
```

Key Points

- Use `threading.Thread(target=func)` to create threads
- `start()` → begins thread execution
- `join()` → waits for thread to finish
- Python threads are **not truly parallel for CPU-bound tasks** due to **GIL**, but ideal for **I/O-bound tasks**
- **Daemon threads** run in the background and exit when the main program ends

Q. Explain Thread synchronization

Ans.

Thread synchronization is the process of coordinating threads to safely access shared resources and prevent race conditions in multithreaded programs.

Using Lock

A **Lock** ensures that **only one thread accesses a shared resource at a time**.

Example:

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    for _ in range(1000):
        lock.acquire()
        counter += 1
        lock.release()

t1 = threading.Thread(target=increment)
t2 = threading.Thread(target=increment)

t1.start()
t2.start()
t1.join()
t2.join()

print("Counter:", counter)
```

Output:

Counter: 2000

Key Points

- `lock.acquire()` → acquires the lock for a thread
- `lock.release()` → releases the lock so other threads can access the resource
- Prevents **race conditions** when multiple threads share data
- Other synchronization primitives: `RLock`, `Semaphore`, `Event`, `Condition`

Q. Explain NoSQL database & Types of NoSQL Ans.

NoSQL Database

A NoSQL (Not Only SQL) database is a non-relational database designed for:

- Flexible schema
- Scalable data storage
- Handling large volumes of unstructured or semi-structured data

Key Points:

- Does not use traditional tables like relational databases
- Often used in big data, real-time web apps, and distributed systems

Type	Description	Examples
Key-Value	Stores data as key-value pairs for fast retrieval	Redis, DynamoDB
Document Store	Stores data as documents (JSON, XML) with flexible schema	MongoDB, CouchDB
Column-Family	Stores data in columns instead of rows , optimized for analytics	Cassandra, HBase
Graph	Stores data as nodes and edges to represent relationships	Neo4j, JanusGraph

Key Points

- **Key-Value** → fast lookup using keys
- **Document** → flexible schema for complex objects
- **Column-Family** → ideal for analytics and large datasets
- **Graph** → ideal for social networks and interconnected data

Q. Explain MongoDB Ans.

MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like documents instead of tables.

- Supports dynamic schema
- Handles large volumes of unstructured or semi-structured data
- Commonly used in web and mobile applications

Term Description

Document Basic unit of data (like a JSON object)

Collection Group of documents (similar to a table in RDBMS)

BSON Binary JSON format used to store documents

MQL MongoDB Query Language for CRUD operations

Features

- **Flexible Schema:** Documents in the same collection can have different fields
- **Indexing:** Improves query performance
- **Sharding:** Horizontal scaling across multiple servers
- **Replication:** Provides high availability and fault tolerance

Example

from pymongo import MongoClient

```
client =
MongoClient("mongodb://localhost:27017/")
db = client["mydatabase"]
collection = db["users"]
```

```
# Insert a document
collection.insert_one({"name": "Amit", "age": 25})
```

```
# Query a document
user = collection.find_one({"name": "Amit"})
print(user)
```

Output:

```
{'_id': ObjectId('...'), 'name': 'Amit', 'age': 25}
```

Output:

```
{'_id': ObjectId('...'), 'name': 'Amit', 'age': 25}
```

Q. Explain SQL Vs NoSQL

Ans.

Feature	SQL (Relational DB)	NoSQL (Non-Relational DB)
Data Model	Tables with rows and columns	Flexible: Key-Value, Document, Column, Graph
Schema	Fixed schema	Dynamic/Flexible schema
Query Language	SQL (Structured Query Language)	Database-specific APIs (e.g., MQL for MongoDB)
Transactions	ACID-compliant, supports complex queries	Usually BASE model; eventual consistency
Scaling	Vertical scaling (scale-up)	Horizontal scaling (scale-out)
Use Case	Financial apps, ERP, complex queries	Big data, real-time web apps, flexible/unstructured data

Q. Explain Installing MongoDB on Windows

Ans.

Steps to Install MongoDB on Windows

- 1. Download Installer**
 - Go to the official MongoDB website and download the **MongoDB Community Edition installer** for Windows.
- 2. Run the Installer**
 - Choose **Complete Setup** to install all components, including MongoDB server, Compass (GUI), and command-line tools.
- 3. Set Up Data Directory**
 - MongoDB stores data in C:\data\db by default.
 - If the folder doesn't exist, **create it manually**.
- 4. Start MongoDB Server**

- Open **Command Prompt** and run:

mongod

5. Open MongoDB Shell

- In another Command Prompt, run:
- mongo
- This opens the MongoDB shell to interact with the database.

6. Optional: Run as Windows Service

- During installation, you can choose to **register MongoDB as a Windows service**.
- This allows MongoDB to **start automatically on system startup**.

7. Verify Installation

- Run mongo in Command Prompt and check the version or connect to the server to ensure it works.

8. GUI Management

- Use **MongoDB Compass** (installed by default) for visual management of databases.

Key Points for Exams:

- MongoDB data directory → C:\data\db
- Server command → mongod
- Shell command → mongo
- Windows service → optional, enables automatic startup

Q. Explain CRUD Operations

Ans.

CRUD Operations

CRUD stands for the **four basic operations performed on a database**:

- **C – Create:** Insert new data
- **R – Read:** Retrieve existing data
- **U – Update:** Modify existing data
- **D – Delete:** Remove data

Operation	Mongo DB Method	Example
Create	insert_one()	collection.insert_one({"name":"Amit", "age":25})
Read	find_one()	collection.find({"name":"Amit"})
Update	update_one()	collection.update_one({"name":"Amit"}, {"\$set":{"age":26}})
Delete	delete_one()	collection.delete_one({"name":"Amit"})

Example:

```
from pymongo import MongoClient
client = MongoClient("mongodb://localhost:27017/")
db = client["mydatabase"]
users = db["users"]

# Create
users.insert_one({"name":"Amit","age":25})

# Read
print(users.find_one({"name":"Amit"}))

# Update
users.update_one({"name":"Amit"}, {"$set":{"age":26}})

# Delete
users.delete_one({"name":"Amit"})
```

Q. Explain Commit, Rollback and Cursor operation

Ans.

1. COMMIT

- **Definition:** Makes all changes in the current transaction **permanent** in the database.
- **Use:** After performing a set of operations (INSERT, UPDATE, DELETE) that you want to **save permanently**.
- **Example (SQL):**

```
BEGIN TRANSACTION;
INSERT INTO students (name, age) VALUES ('Amit', 20);
COMMIT; -- Saves the changes permanently
```

2. ROLLBACK

- **Definition:** **Undoes all changes** made in the current transaction, reverting the database to the previous stable state.
- **Use:** When an **error occurs** or conditions are not met, you can rollback to prevent partial changes.
- **Example (SQL):**

```
BEGIN TRANSACTION;
UPDATE students SET age = 21 WHERE name='Amit';
ROLLBACK;
```

3. CURSOR

- **Definition:** A **pointer to a specific row** in a result set, allowing **row-by-row processing** of query results.
- **Use:** Useful for **processing large datasets** or iterating through query results.
- **Example (SQL/Python):**

```
import sqlite3
conn = sqlite3.connect('mydb.db')
cursor = conn.cursor()
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)
```

Key Points

- **COMMIT:** Makes changes permanent
- **ROLLBACK:** Undoes changes in the transaction
- **CURSOR:** Used to navigate and process result sets efficiently

Q. Explain Django Project Structure and Django Models

Ans.

When you create a Django project using `django-admin startproject projectname`, the structure looks like:

```
projectname/
|
├── manage.py
├── projectname/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── appname/
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── views.py
│   └── migrations/
```

Key Points for Exams:

- `manage.py` → used to runserver, migrate, shell, startapp
- `settings.py` → stores database, installed apps, middleware
- `urls.py` → maps URL patterns to view functions
- `wsgi.py` → deployment entry point

Django Models

A model is a Python class that represents a database table. Each attribute of the class corresponds to a database column.

Example:

```
from django.db import models
```

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
```

```
def __str__(self):
    return self.name
```

Key Points:

- Models are defined in `models.py` of each app
- `CharField`, `IntegerField`, `EmailField` → define column types
- Use Django ORM methods:
 - `Student.objects.all()` → retrieve all records
 - `Student.objects.create(name="Amit", age=20, email="amit@example.com")` → create record
- After defining models, run:
 - `python manage.py makemigrations`
 - `python manage.py migrate`

Conclusion:

- **Project structure** organizes settings, URLs, apps, and static files
- **Models** define the data structure and interact with the database using Django ORM

Q. Explain Django Views and Django Templates

Ans.

Django Views

A **view** is a **Python function or class** that handles web requests and returns a **response** (HTML, JSON, or redirect).

Key Points:

- Defined in `views.py` of each app
- Processes data, interacts with models, and passes context to templates
- Returns an `HttpResponse` or rendered template

Example:

```
from django.shortcuts import render
from .models import Student
```

```
def student_list(request):
    students = Student.objects.all() # Fetch data from model
    return render(request, 'students.html',
                  {'students': students})
```

Django Templates

A **template** is an **HTML file** that defines the **presentation layer**. It can display dynamic data passed from views.

Key Points:

- Stored in a templates directory within app or project
- Use **template tags** and **variables** to render dynamic content
- Separates **presentation logic from Python code**

Example

```
<!DOCTYPE html>
<html>
<head>
  <title>Students</title>
</head>
<body>
  <h1>Student List</h1>
  <ul>
    {% for student in students %}
      <li>{{ student.name }} - {{ student.age }}</li>
    {% endfor %}
  </ul>
</body>
</html>
```

Interaction Between Views and Templates

1. **View** retrieves data (usually from models)
2. **View** passes data via a **context dictionary**
3. **Template** renders the data for display

Example Interaction:

```
# View
return render(request, 'template.html', {'key':
value})
```

```
# Template
{{ key }} # Outputs the value in HTML
```

Q. Explain Django URLs and Django Forms Ans.

Django **URLs** map **web addresses to view functions** that handle the request and return a response.

Key Points:

- Defined in urls.py of a project or app
- Use path() to map a URL to a view
- Use include() to include URLs from another app for modular design

Example (urls.py):

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('students/', views.student_list,
name='student_list'),
]
```

Django Forms

A **Django Form** is a **Python class** that handles **HTML form rendering, validation, and data submission**.

Key Points:

- Defined in forms.py (or django.forms)
- Can be **Form** (manual fields) or **ModelForm** (linked to a database model)
- Use is_valid() to check submitted data
- Use form.save() to save data for ModelForms

Example:

```
from django import forms
from .models import Student
```

```
class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'age', 'email']
```

Interaction

1. **URL** → maps request to view
2. **View** → processes form data, validates it
3. **Form** → renders HTML and handles submission
4. **Template** → displays form and errors to the user

Q. Explain Django Authentication and Advanced Django Features

Ans.

Django Authentication is a built-in system that manages user accounts, login, logout, and permissions.

Key Points:

- Uses django.contrib.auth module
- Includes the **User model**, groups, permissions, and authentication functions

Example:

```
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import redirect
```

```
def user_login(request):
    username = request.POST['username']
    password = request.POST['password']
    user = authenticate(request,
username=username, password=password)
    if user is not None:
        login(request, user)
        return redirect('home')
    else:
        return redirect('login')
```

Advanced Django Features

Feature	Purpose
Signals	Allow decoupled apps to be notified when events occur (e.g., post_save)
Middleware	Process requests and responses globally, e.g., authentication, logging
Custom Managers	Add reusable query logic or table-level functionality in models
Caching	Store frequently accessed data in memory to reduce database queries
Decorators	Modify behavior

Q. Explain Django Rest Framework (DRF) Ans.

DRF is a powerful toolkit to build Web APIs in Django. It allows developers to create RESTful APIs easily and provides serialization, authentication, and routing out of the box.

Component	Purpose
Serializer	Converts Django models or Python data into JSON and validates input data
ModelSerializer	Automatically maps Django model fields to serializer fields
APIView	Handles HTTP methods like GET, POST, PUT, DELETE
ViewSet	Combines list, create, retrieve, update, delete actions in one class
Router	Automatically generates URL patterns for ViewSets
Response	Returns JSON responses to clients
Authentication	Supports Token, Session, JWT, etc. for API access
Permissions	Controls access to API endpoints (e.g., IsAuthenticated)

Example of DRF:

```
from rest_framework import serializers
from .models import Student

class
StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = ['id', 'name', 'age', 'email']
```

Advantages of DRF

- Simplifies **API development** in Django
- Provides **serialization** and **validation**
- Built-in **authentication and permissions**
- Supports **ViewSets and Routers** for concise code

Q. Explain Deployment and Performance Optimization

Ans.

Deployment in Django

Deployment is the process of **making a Django application live** on a server so that users can access it.

Key Points:

- Use a **WSGI server** like **Gunicorn** or **uWSGI** to run the Django app
- **Nginx** often acts as a **reverse proxy** and serves static/media files
- Use **virtual environments** to isolate project dependencies

Static and media files should be collected for production:

```
python manage.py collectstatic
```

Performance Optimization

Improving speed, scalability, and responsiveness of Django applications.

Technique	Description
Caching	Store frequently accessed data in memory using Memcached or Redis to reduce database queries
Database Optimization	Use <code>select_related</code> and <code>prefetch_related</code> to minimize queries
Load Balancing	Distribute traffic across multiple servers for high traffic websites
Middleware Optimization	Remove unnecessary middleware to reduce request processing time
Serving Static/Media Separately	Use Nginx or a CDN to offload static content from Django
Monitoring Tools	Use tools like New Relic , Sentry , Prometheus to track performance, errors, and latency

Q. Explain Building a real-world Django application with Django Channels for WebSockets

Ans.

Django Channels Overview

Django Channels extends Django to support real-time, asynchronous applications using WebSockets alongside traditional HTTP.

Key Points:

- Handles WebSocket connections for live data updates
- Uses ASGI (Asynchronous Server Gateway Interface) instead of WSGI
- Enables chat apps, notifications, live feeds, and multiplayer games

Component	Purpose
ASGI Application	Entry point for async requests, required for Channels
Consumer	Python class handling WebSocket events (connect, receive, disconnect)
Channel Layer	Manages groups of connections and broadcasting messages
routing.py	Maps WebSocket URL paths to Consumers
Daphne / runserver	ASGI server to run the Channels app

Example:

```
from channels.generic.websocket import AsyncWebsocketConsumer
import json
```

```
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        await self.accept()

    async def receive(self, text_data):
        data = json.loads(text_data)
        message = data['message']
        await
        self.send(text_data=json.dumps({'message':
            message}))

    async def disconnect(self, close_code):
        pass
```

